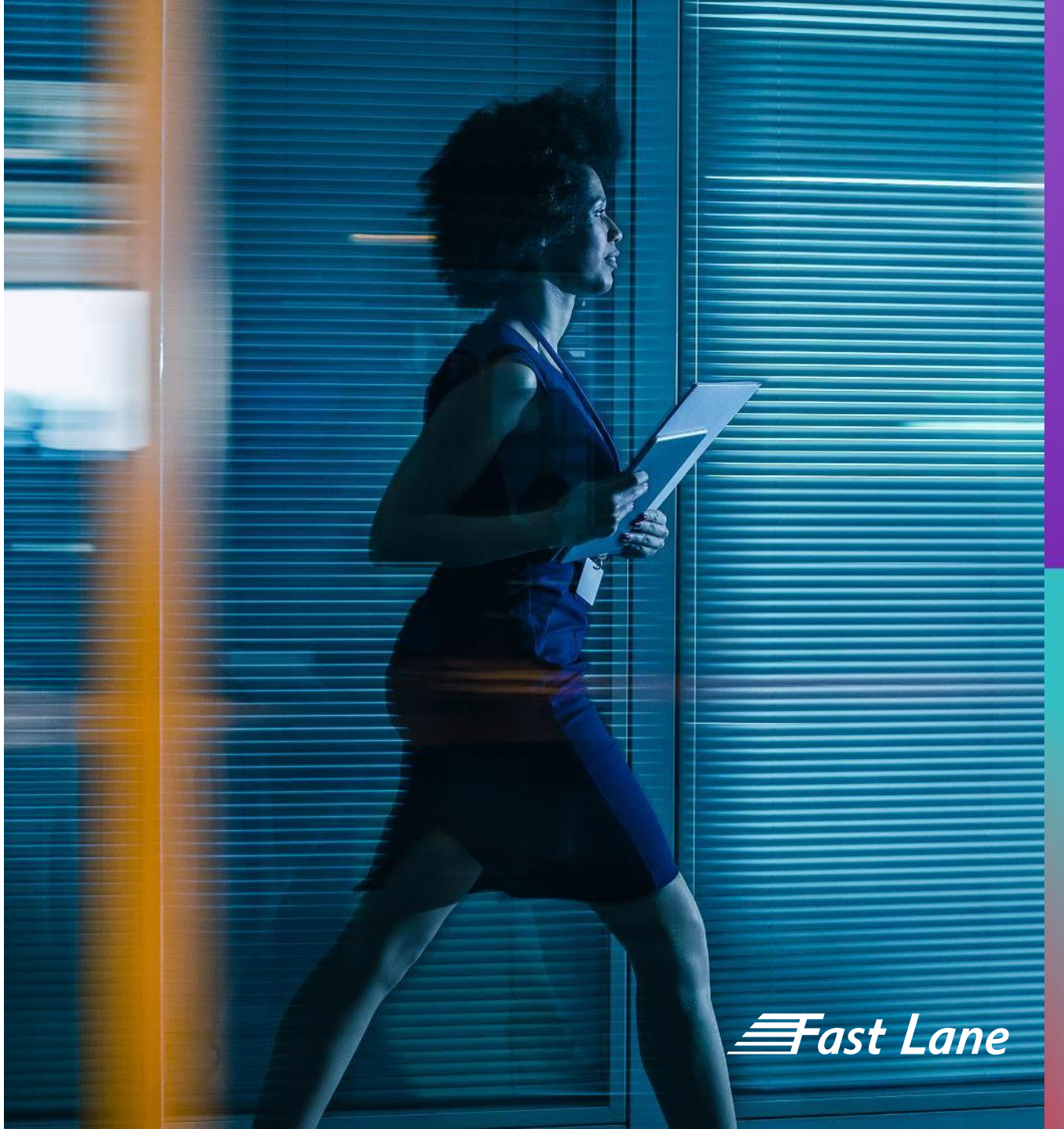
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 4

Reliable Messaging with Service Bus

Parcel events from the shipping API and scanners to their consumers

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Topic Subscriptions: SQL Filters

Correlation and SQL filters, rule actions and Bicep

02

Azure Functions: Service Bus Triggers

Isolated worker trigger, settlement and host.json

03

Competing Consumers: KEDA Scale Rules

ServiceBusProcessor concurrency and KEDA scaling

04

Sessions: Ordered Message Processing

Session IDs, session state and session locks

05

Dead-Letter Queues: Poison Messages

Delivery limits, reasons and resubmitting

06

Idempotent Consumers: Duplicate Detection

Duplicate detection and a Cosmos DB marker

Reliable messaging with Service Bus

Parcel events from the shipping API and scanners to their consumers

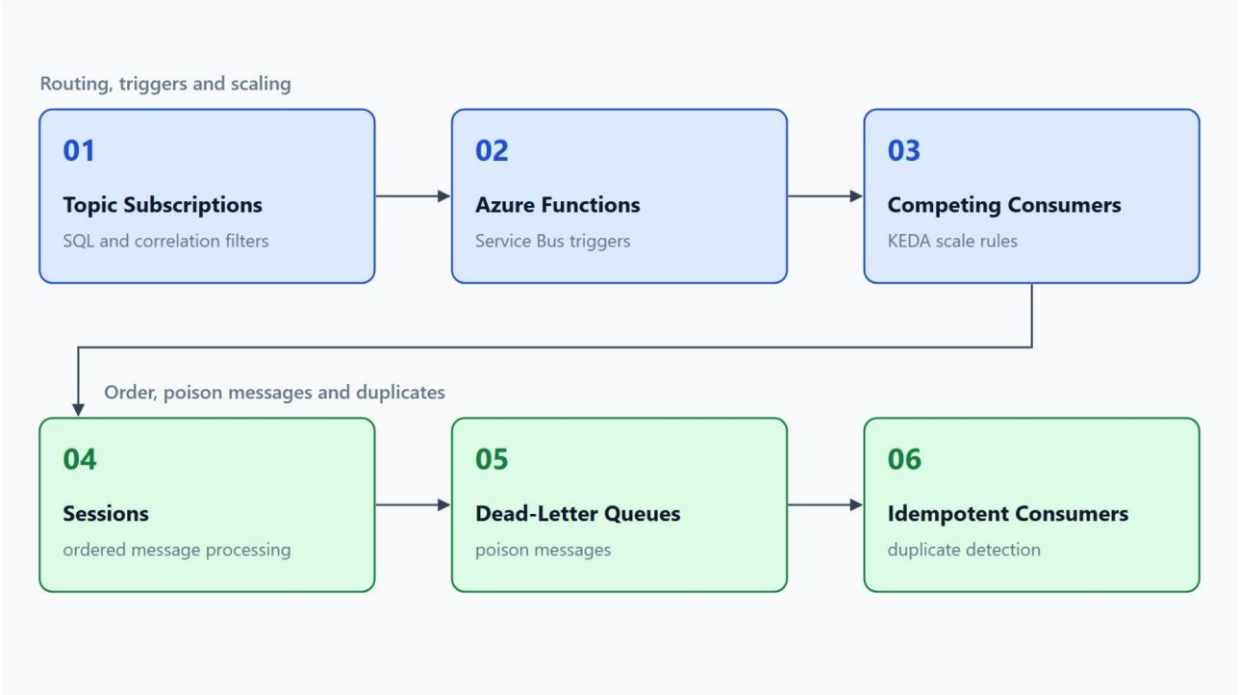
- **Routing and consuming**: filtered subscriptions, a Functions trigger and consumers scaled by KEDA
- **Order and poison messages**: one session per parcel, unreadable label scans in the dead-letter queue
- **Duplicates**: duplicate detection at the broker and an idempotent consumer



Overview

Six reliability features for the parcel event flow

- Route each parcel event to the **subscriptions** whose filters select it
- Consume `tracking` with a **Functions trigger** that settles after its work
- Scale **competing consumers** with a KEDA rule on the backlog
- Keep each parcel's scans in order with **sessions**
- Dead-letter **poison messages** and drop duplicates with an idempotent consumer



01

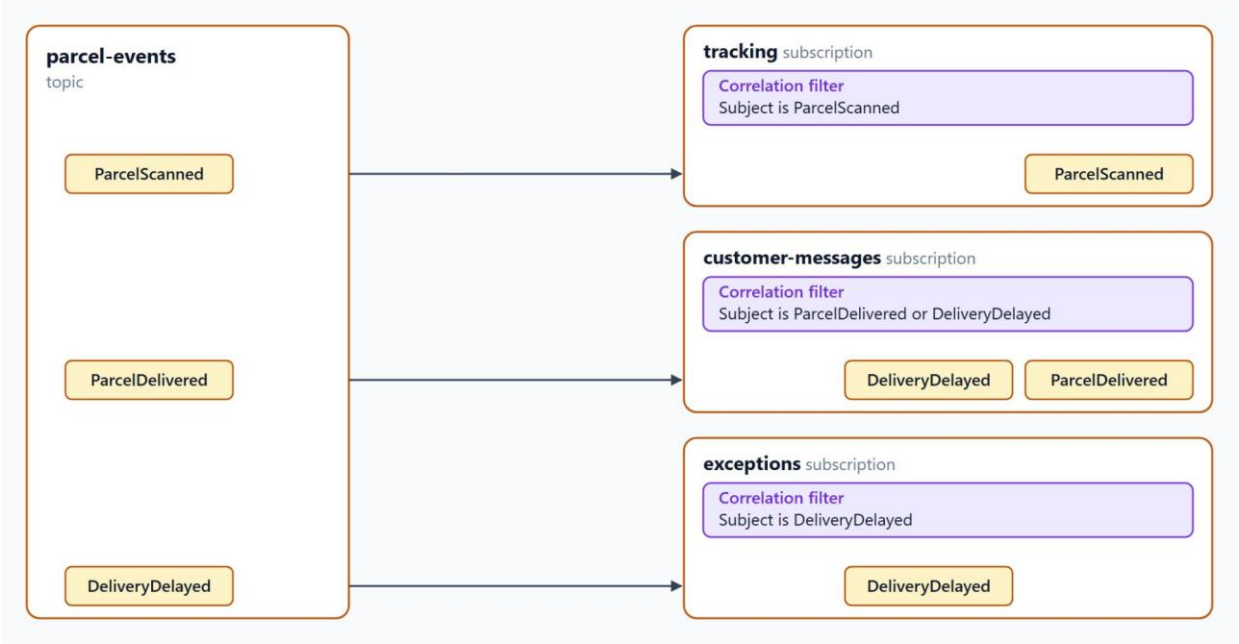
Topic Subscriptions: SQL Filters

Correlation and SQL filters, rule actions and Bicep

Topics, subscriptions and filters

Each subscription keeps its own copy of the events it selects

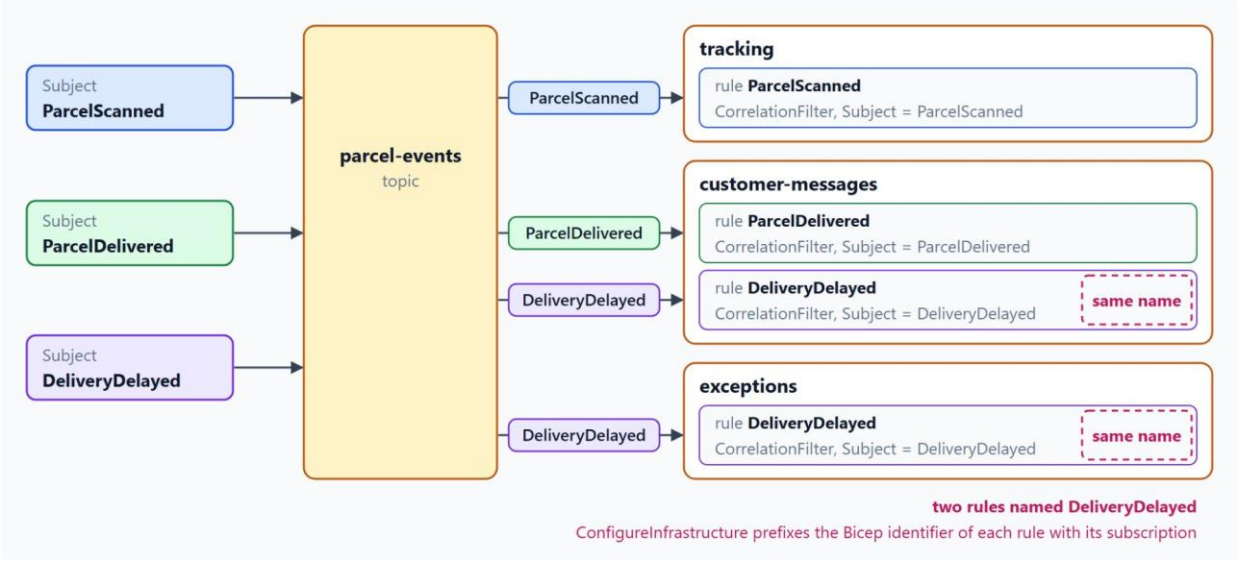
- Each **subscription** receives a copy of every message its rules match
- `tracking` gets `ParcelScanned`, `customer-messages` gets `ParcelDelivered` and `DeliveryDelayed`, `exceptions` gets `DeliveryDelayed`
- A slow consumer builds a backlog on its own subscription only
- On Azure, every new subscription starts with a **default rule** that matches every message
- Rules on one subscription combine with OR, so delete the default rule



Subscription rules in code and Bicep

Correlation filters on Subject for every Contoso Logistics subscription

- **Correlation filters** match fixed properties exactly; Microsoft Learn recommends them whenever they fit
- SQL filters add `LIKE`, `IS NULL` and `EXISTS` over properties, never the message body
- A rule action runs `SET` or `REMOVE` on the copy the subscription receives
- `ServiceBusAdministrationClient.CreateRuleAsync` creates rules in code
- The AppHost declares `AzureServiceBusRule`, and Aspire generates the Bicep



02

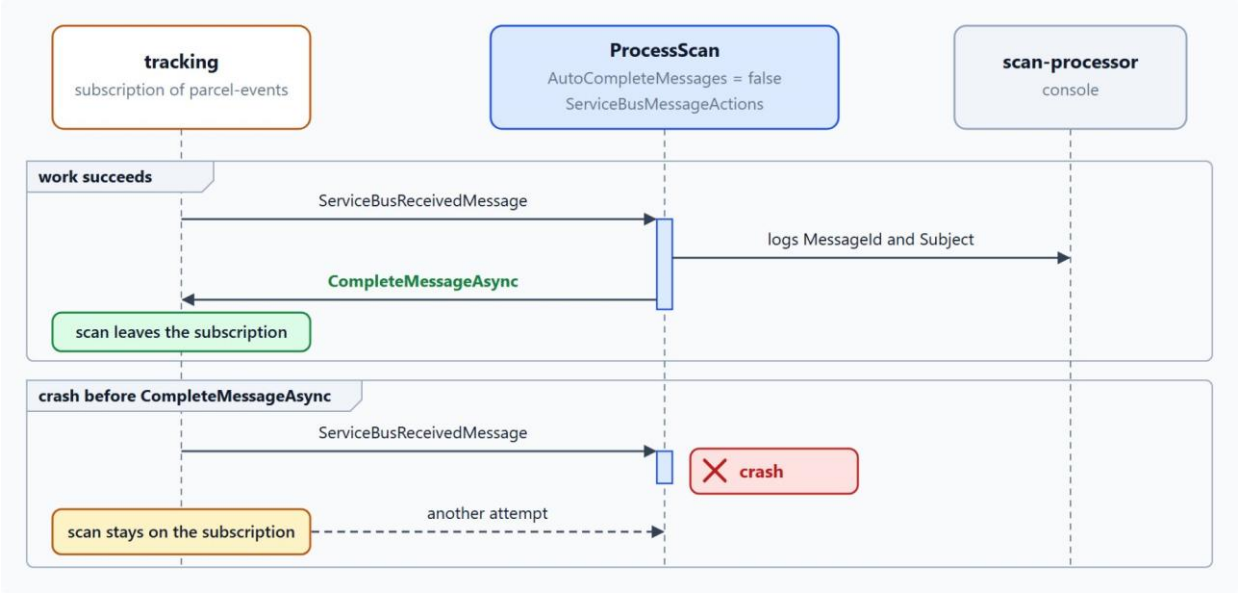
Azure Functions: Service Bus Triggers

Isolated worker trigger, settlement and host.json

Service Bus trigger and settlement

The function completes a scan only after its work succeeds

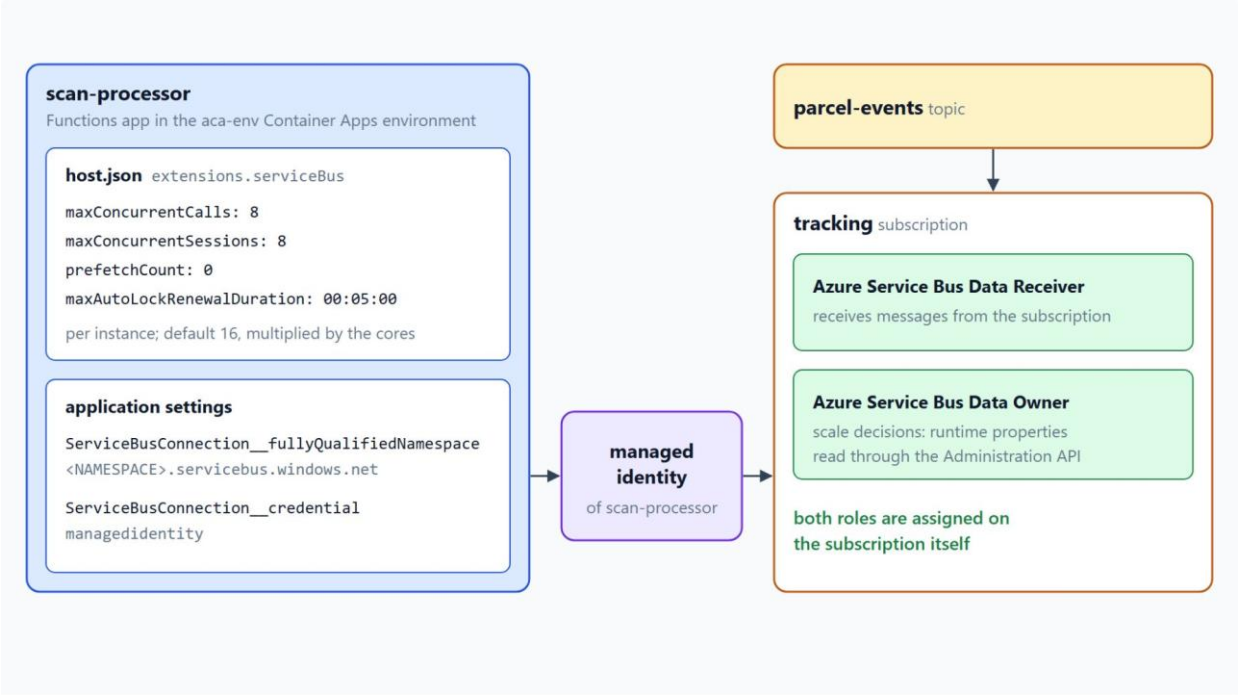
- Isolated worker model with `Microsoft.Azure.Functions.Worker.Extensions.ServiceBus` 5.14.1 or later
- Bind `ServiceBusReceivedMessage` for metadata and `ServiceBusMessageActions` to settle
- Set `AutoCompleteMessages = false` whenever the function binds message actions
- **Settlement**: complete on success, abandon to retry, dead-letter a poison message
- An unhandled exception or an expired lock returns the message with a higher delivery count



Concurrency, connections and hosting

The trigger connects through the app's managed identity

- `maxConcurrentCalls` limits callbacks per instance: 16 by default, multiplied by the cores
- `maxConcurrentSessions` 8 caps the parcel sessions one instance works on
- `ServiceBusConnection__fullyQualifiedNamespace` and `__credential` replace the connection string
- **Roles on the subscription**: Data Receiver to receive, Data Owner for scale decisions
- `AddAzureFunctionsProject` places `scan-processor` in the `aca-env` Container Apps environment



03

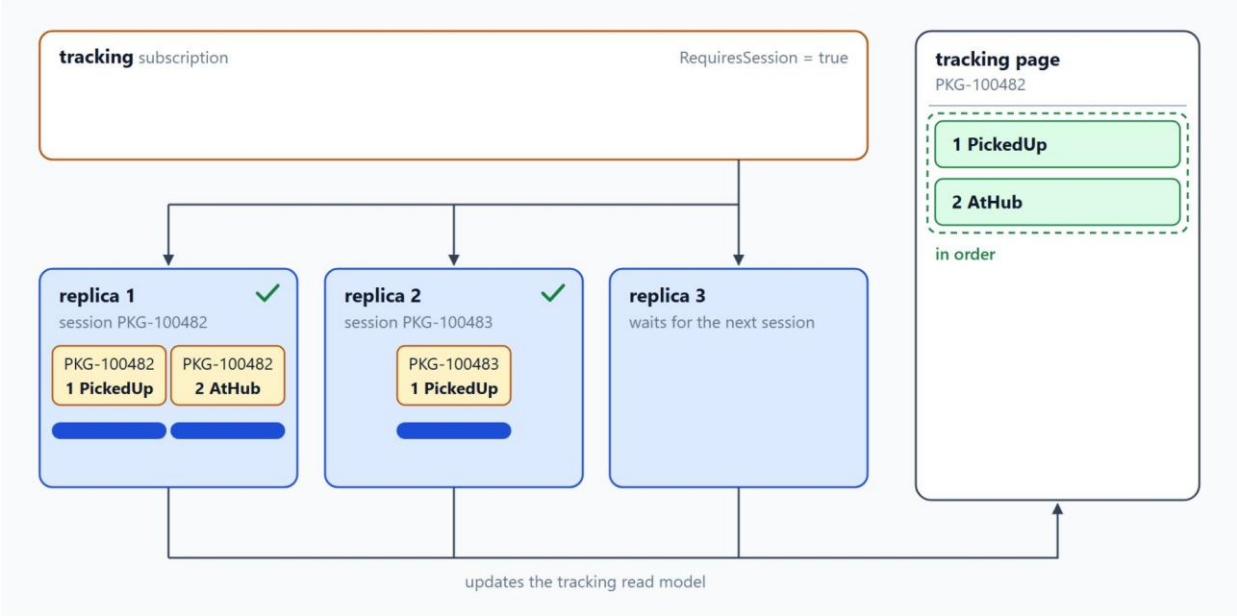
Competing Consumers: KEDA Scale Rules

ServiceBusProcessor concurrency and KEDA scaling

Competing consumers

Replicas share the tracking subscription, and sessions keep each parcel in order

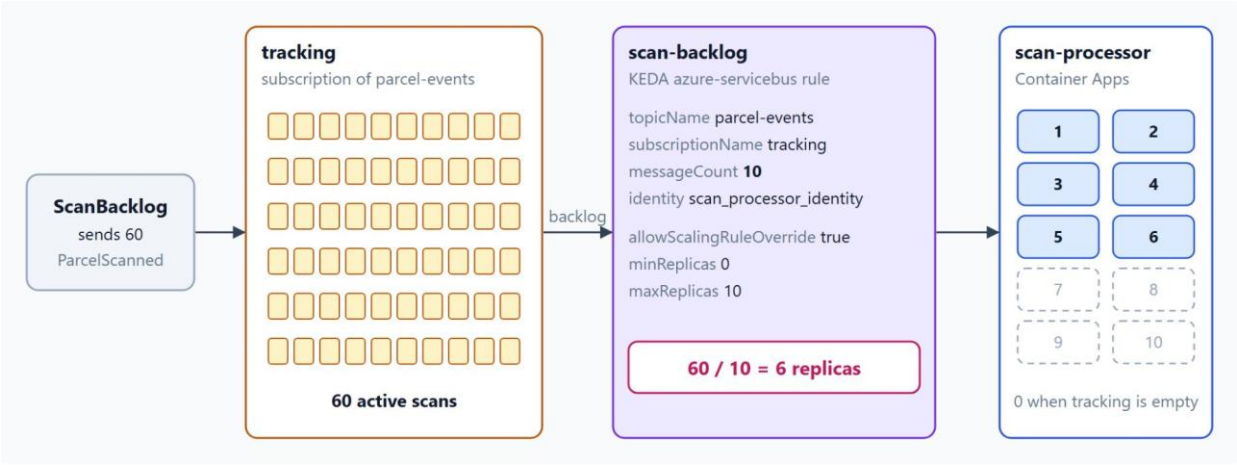
- Several instances receive from one subscription, and each message reaches one of them
- More instances raise throughput, and a failed instance doesn't block the others
- Parallel consumers give up ordering, so `tracking` requires **sessions**
- `ServiceBusProcessor`: `MaxConcurrentCalls` defaults to 1, `PrefetchCount` to 0
- Set `PrefetchCount` at least as high as `MaxConcurrentCalls`, and keep it small for slow handlers



KEDA scale rule in Container Apps

Replicas follow the backlog of the tracking subscription

- A custom `azure-servicebus` rule reads the depth of the `tracking` subscription
- **messageCount** sets the messages per replica: 5 by default, 10 in the lab
- The rule's `identity` reads the backlog with a managed identity instead of a connection string
- `minReplicas` 0 scales to zero, and `maxReplicas` caps the cost
- 60 waiting scans with `messageCount` 10 give 6 of the 10 replicas



04

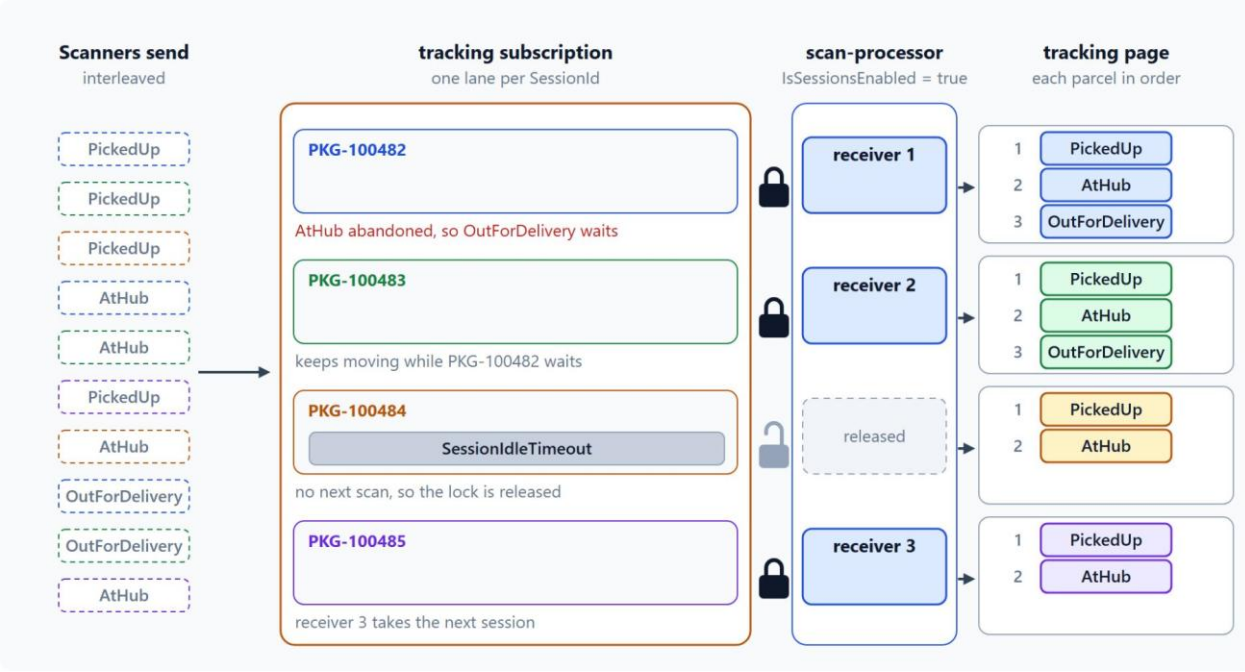
Sessions: Ordered Message Processing

Session IDs, session state and session locks

Session-enabled subscriptions

The tracking number is the session ID, so each parcel gets one ordered lane

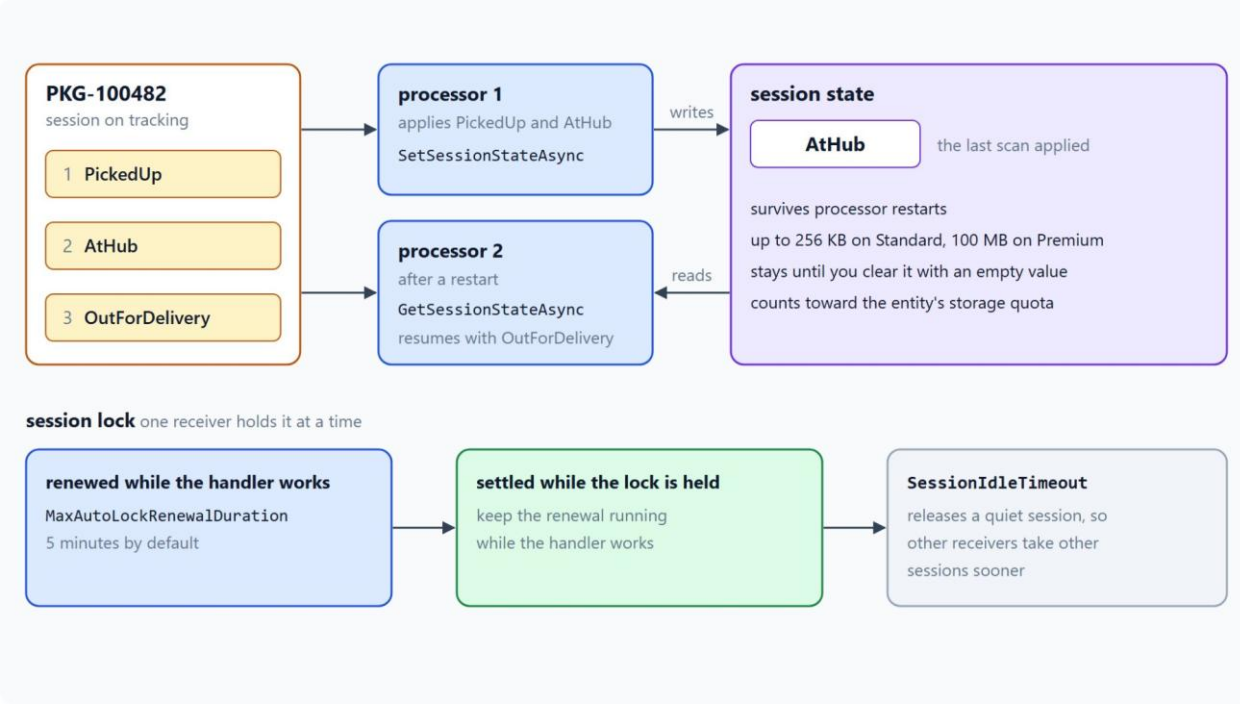
- `RequiresSession`: messages that share a `SessionId` form a first-in, first-out sequence
- The tracking number is the **session ID**, so every scan of a parcel shares a session
- Different sessions run in parallel, and one receiver at a time holds a session's lock
- An abandoned scan holds back only the later scans of its own parcel
- A message without a session ID dead-letters with `Session ID is null`



Session state and session locks

A checkpoint that survives restarts and a lock renewed while the handler works

- `ServiceBusSessionProcessor` works on up to `MaxConcurrentSessions` sessions, 8 by default
- **Session state** is a checkpoint that survives processor restarts
- The processor renews the lock up to `MaxAutoLockRenewalDuration`, 5 minutes by default
- `SessionIdleTimeout` closes a quiet session, so the processor moves on to another
- Sessions need the Standard or Premium tier



05

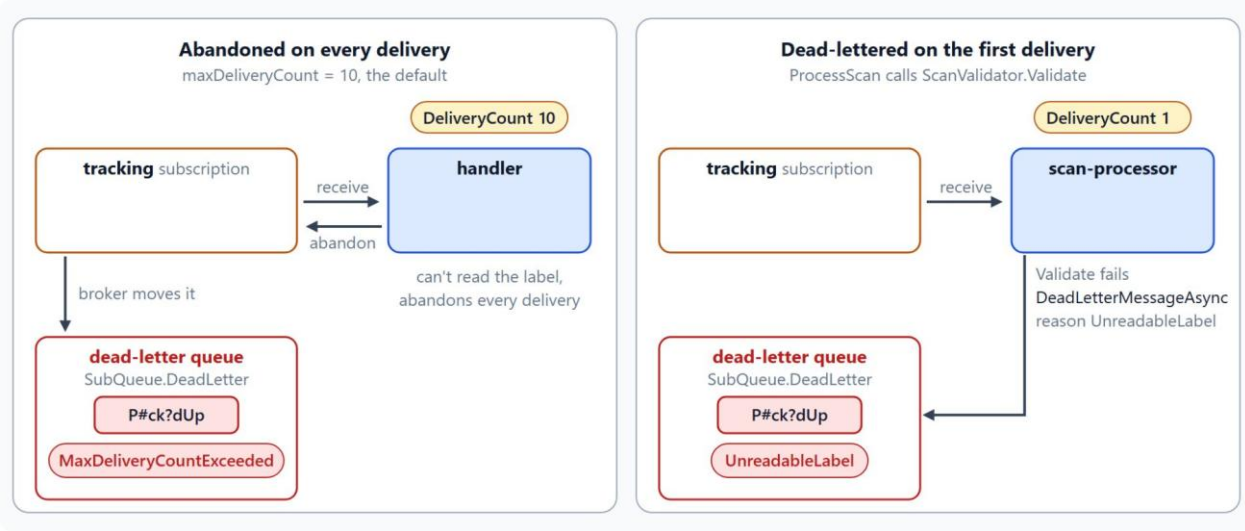
Dead-Letter Queues: Poison Messages

Delivery limits, reasons and resubmitting

Delivery limit and dead-letter reasons

The broker moves a message once its delivery count passes maxDeliveryCount

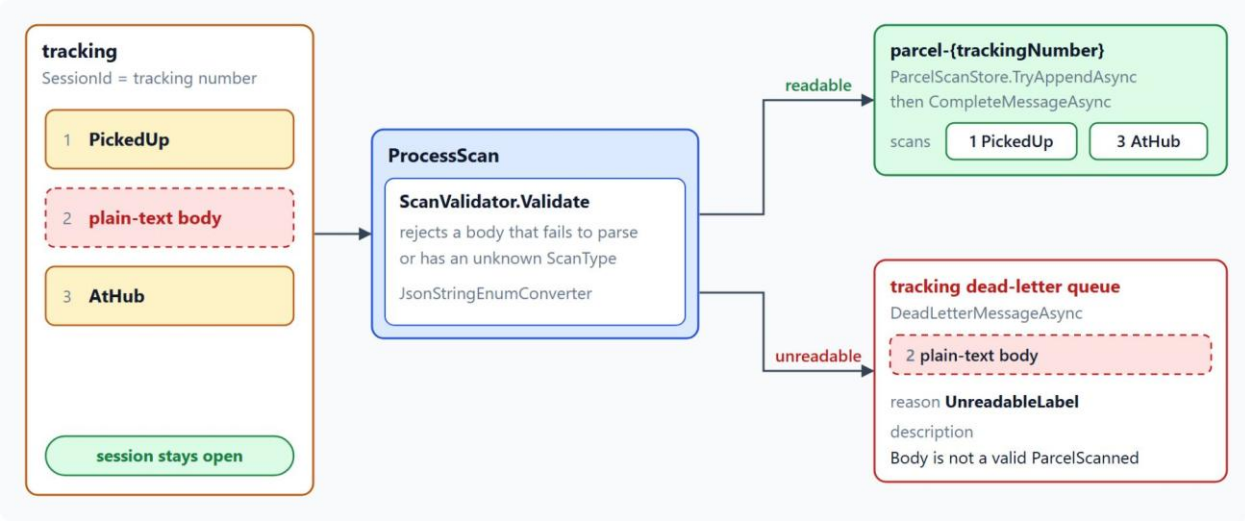
- Every abandon and every expired lock adds one to the **delivery count**
- Past `maxDeliveryCount`, 10 by default, the broker dead-letters with `MaxDeliveryCountExceeded`
- Other system reasons: `TTLExpiredException`, `HeaderSizeExceeded`, `Session ID is null`, `MaxTransferHopCountExceeded`
- `deadLetteringOnMessageExpiration` moves every expiring message to the dead-letter queue
- Dead-letter a scan yourself on its first delivery when it can't succeed



Dead-lettering and resubmitting unreadable scans

The reason travels with the message while the parcel's other scans keep moving

- `DeadLetterMessageAsync` takes a reason and a description, each up to 4,096 characters
- Use a short, stable **reason code** such as `UnreadableLabel`
- Only that message moves, so the parcel's session stays open
- Read dead letters with `SubQueue.DeadLetter` and their `DeadLetterReason`
- Send a repaired copy first, then complete the original



06

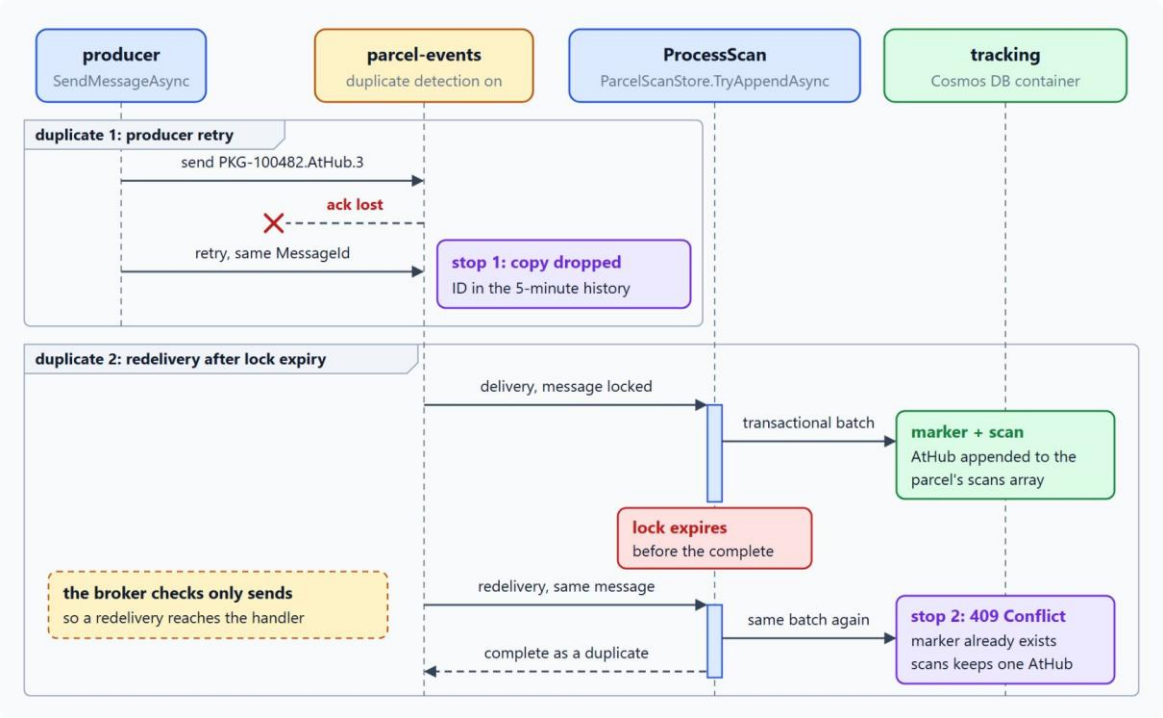
Idempotent Consumers: Duplicate Detection

Duplicate detection and a Cosmos DB marker

Duplicate detection at the broker

A stable MessageId lets the broker drop a producer retry

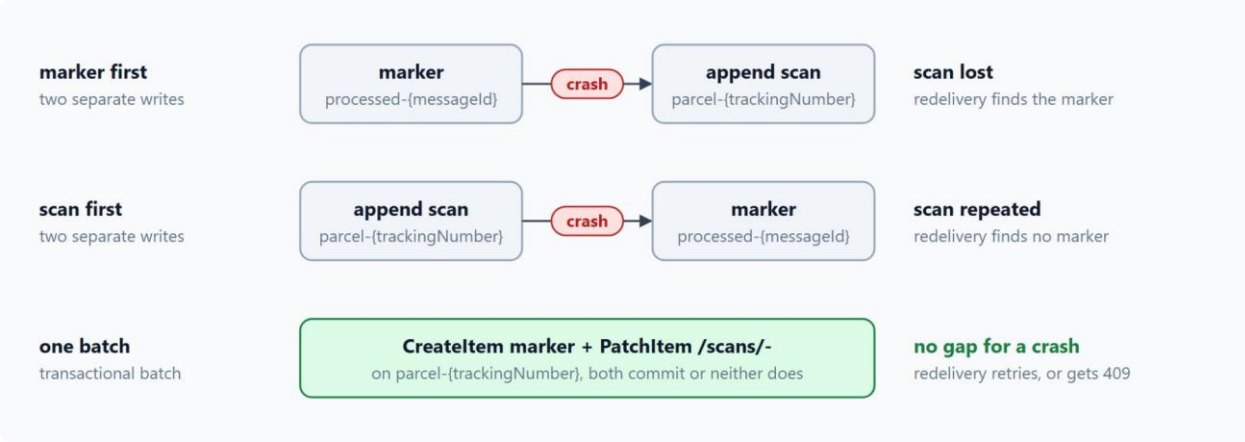
- Service Bus delivers each message **at least once**, so the same scan can arrive twice
- The broker keeps `MessageId` values for a window: 10 minutes by default, 20 seconds to 7 days
- The `parcel-events` topic uses a five-minute window
- A stable `MessageId` such as `PKG-100482.AtHub.3` makes a retry match
- The broker checks only sends, so a redelivery still reaches the handler



Marker and scan in one transactional batch

Cosmos DB commits both writes or neither

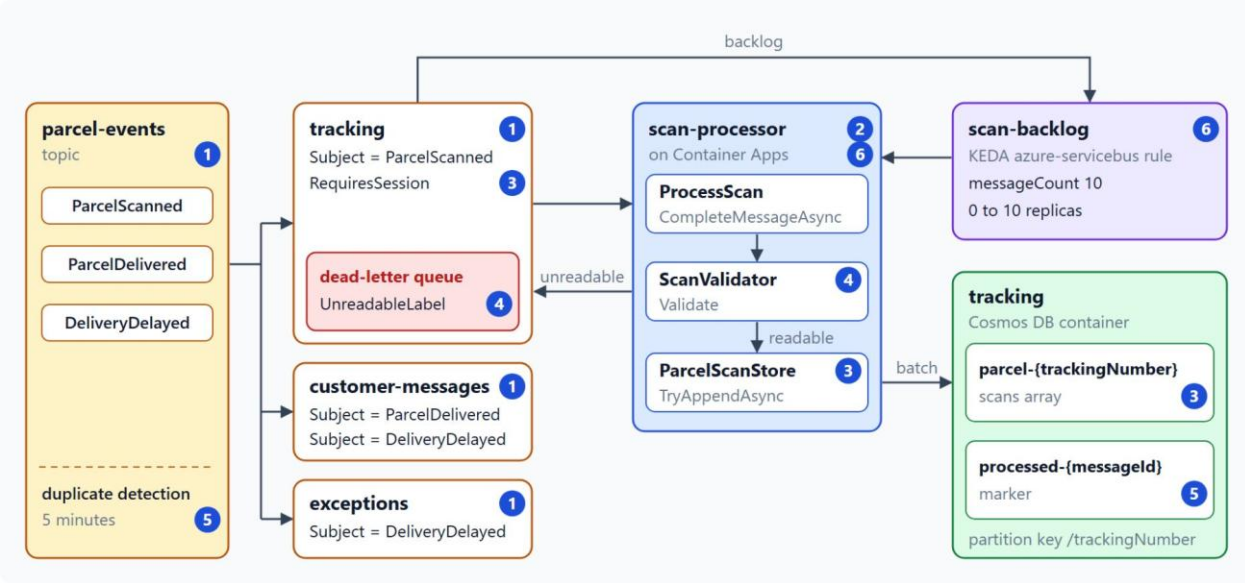
- The **Idempotent Consumer** pattern records each processed key in a store that enforces uniqueness
- Cosmos DB answers a repeated item ID with a `CosmosException` whose status is `Conflict` (409)
- One transactional batch creates `processed-{messageId}` and patches `parcel-{trackingNumber}`
- Marker and parcel share the `/trackingNumber` partition, up to 100 operations and 2 MB
- A 409 on the marker completes the message as a duplicate; any other failure retries



The scan processing lab

Seven steps from the parcel-events topic to a consumer that scales on its backlog

- Steps 1 and 2: `parcel-events` with three filtered subscriptions, and a **Functions trigger** that settles after its work
- Step 3: sessions on `tracking` keep each parcel's scans in order
- Step 4: unreadable scans dead-letter with the reason `UnreadableLabel`
- Step 5: duplicate detection and a marker in one Cosmos DB batch
- Steps 6 and 7: deploy once, scale with KEDA, record the patterns in `docs/scan-processing.md`



Takeaways

What to carry into the Event Grid and Event Hubs module

- Give each consumer a filtered subscription and delete the default rule
- Settle after the work: `AutoCompleteMessages = false` with message actions
- Scale on the backlog with a KEDA rule and a managed identity
- Use the parcel ID as the session ID to keep each parcel's scans in order
- Dead-letter unreadable scans with a reason, and write each scan once with a marker batch

